

The Source of Truth – Git and GitHub

The Version Control Problem

In traditional network engineering, configuration files and templates are often stored on local hard drives or shared network drives. This leads to a common operational challenge: tracking file versions. It is standard practice to see directories filled with files named `config_v1.txt`, `config_v2_final.txt`, or `config_v3_edited.txt`.

This methodology makes it nearly impossible to audit changes over time, understand why a specific change was made, or safely roll back to a previous state when multiple administrators are working on the same network. To solve this, the industry relies on Version Control Systems (VCS).

Git: The Local Engine

Git is a free, open-source software application installed locally on an administrator's machine. It operates as the underlying engine for version control.

Git does not store files in a traditional sense; rather, it tracks the *changes* made to files over time. When a local directory is initialized with Git, it becomes a "Repository." From that point forward, Git monitors every addition, deletion, or modification made to the files within that folder. It operates entirely locally and does not require an internet connection to function.

GitHub: The Collaborative Platform

While Git is the local software engine, **GitHub** is a centralized, cloud-based hosting platform. (Other platforms, such as GitLab or Bitbucket, serve the exact same function).

If Git allows an individual to track versions locally, GitHub enables enterprise-wide collaboration. It provides a centralized web interface where teams can store their Git repositories, back up their configurations, and interact with other systems.

The Git Lifecycle

Core Operations To move files between the local Git engine and the centralized GitHub platform, administrators utilize a specific set of operations. Understanding this lifecycle is critical for managing Infrastructure-as-Code:

1. **Clone:** This is the act of copying an existing repository from GitHub down to a local machine. It creates a local workspace linked to the central source.
2. **Stage (Add):** When local files are modified, they are not automatically saved to the version history. Staging is the process of selecting exactly which modified files should be included in the next version snapshot.
3. **Commit:** A commit is the permanent snapshot of the staged files. Unlike simply saving a file, a commit requires a "Commit Message"—a plain-English description of what was changed and why. This ensures the history of the file inherently becomes its documentation.
4. **Push:** Once commits are made locally, they must be uploaded to the centralized repository. Pushing sends the local commits up to GitHub so the rest of the team can view them.
5. **Pull (Sync):** If another administrator makes changes and pushes them to GitHub, other team members must download those updates to keep their local workspaces current. Pulling fetches the latest changes from GitHub and merges them into the local repository.

Abstracting Complexity

Natively, Git is a Command Line Interface (CLI) tool. Executing the lifecycle operations traditionally requires memorizing complex syntax (e.g., `git commit -m "Updated BGP neighbors"`). For users unfamiliar with software development, this CLI can present a steep learning curve.

However, modern Integrated Development Environments (IDEs) like Visual Studio Code abstract this complexity. VS Code features a built-in "Source Control" graphical interface that allows users to stage, commit, push, and pull files with simple button clicks. This graphical abstraction removes the barrier to entry, allowing network administrators to leverage the full power of Git without needing to memorize CLI commands.

How They Interoperate: The Single Source of Truth Git and GitHub do not compete; they interoperate to form a complete workflow. An administrator writes code in an IDE, uses Git to track and commit those changes locally, and then pushes those commits to GitHub for peer review and deployment.

By combining the local tracking capabilities of Git with the centralized hosting of GitHub, the repository becomes the authoritative state of the network—the Single Source of Truth. If a router's running configuration deviates from the code stored in the Main branch on GitHub,

the router is considered out of compliance. This paradigm ensures that all network changes are documented, reviewed, and auditable.